NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

GONÇALO GOMES RODRIGUES

BSc in Software Engineering Degree

THE UNOBSERVABILITY SCHEDULER

USE OF DIFFERENTIAL PRIVACY FOR UNOBSERVABLE PRIVACY-PRESERVED COMMUNICATION

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon
month, year

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

THE UNOBSERVABILITY SCHEDULER

USE OF DIFFERENTIAL PRIVACY FOR UNOBSERVABLE PRIVACY-PRESERVED COMMUNICATION

GONÇALO GOMES RODRIGUES

BSc in Software Engineering Degree

Adviser: Henrique João
Full Professor, NOVA University Lisbon

Co-advisers: João Vilalonga
Ph.D Student, NOVA University Lisbon

Hugo Pereira
Ph.D Student, NOVA University Lisbon

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon
month, year

Abstract

The internet has become an indispensable tool for accessing information and exercising freedom of speech, a fundamental right that empowers individuals worldwide. However, authoritarian regimes and government agencies tend to suppress this right through surveillance and censorship by deploying increasingly sophisticated techniques, such as traffic analysis attacks, to monitor and undermine anonymity, placing users' privacy at significant risk. In response to the growing demand for secure and private communication, anonymity networks like Tor have emerged as essential tools for safeguarding online privacy and fighting censorship, which millions of people rely on daily. Yet, vulnerabilities in Tor's design, particularly against fingerprinting and correlation attacks, expose critical weaknesses that break its effectiveness.

This dissertation addresses these challenges by strengthening Tor Network's defences against the abovementioned attacks. The main goal is to develop an innovative solution that enhances Tor's existing architecture by introducing formal privacy guarantees. The solution is designed to maintain practical levels of throughput and latency, ensuring compatibility with real-world usage scenarios while providing users with adaptable and robust privacy protections.

At the core of the proposed solution is the integration of Differential Privacy principles to introduce carefully calibrated noise into traffic flows to protect users' privacy, which mitigates correlation attacks and empowers users with customizable privacy controls, allowing them to balance privacy and utility according to their needs. To the best of our knowledge, this work represents a pioneering effort in incorporating Differential Privacy into the Tor Network. It delivers a formal, scalable, and effective mechanism that bolsters online privacy, resists state-of-the-art censorship techniques, and upholds the fundamental right to free expression in an increasingly monitored digital world.

Keywords: Anonymity Networks, Differential Privacy, Privacy-Preserving Communication, Covert Communication, Traffic Analysis Attacks

Resumo

A Internet tornou-se uma ferramenta indispensável para aceder à informação e exercer a liberdade de expressão, um direito fundamental que dá poder aos indivíduos em todo o mundo. No entanto, os regimes autoritários e as agências governamentais tendem a suprimir este direito através da vigilância e da censura, recorrendo a técnicas cada vez mais sofisticadas, como os ataques de análise de tráfego, para monitorizar e comprometer o anonimato, colocando a privacidade dos utilizadores em risco. Em resposta à crescente procura de comunicações seguras e privadas, as redes de anonimato, como o Tor, surgiram como ferramentas essenciais para salvaguardar a privacidade em linha e combater a censura, nas quais milhões de pessoas confiam diariamente. No entanto, as vulnerabilidades na conceção do Tor, particularmente contra ataques de fingerprinting e de correlação, expõem fraquezas críticas que quebram a sua eficácia.

Esta dissertação aborda estes desafios, reforçando as defesas da rede Tor contra os ataques acima referidos. O principal objetivo é desenvolver uma solução inovadora que melhore a arquitetura existente do Tor, introduzindo garantias formais de privacidade. A solução foi concebida para manter níveis práticos de débito e latência, assegurando a compatibilidade com cenários de utilização reais, ao mesmo tempo que fornece aos utilizadores protecções de privacidade adaptáveis e robustas.

No centro da solução proposta está a integração dos princípios da Privacidade Diferencial para introduzir ruído cuidadosamente calibrado nos fluxos de tráfego para proteger a privacidade dos utilizadores, o que mitiga os ataques de correlação e dá aos utilizadores controlos de privacidade personalizáveis, permitindo-lhes equilibrar a privacidade e a utilidade de acordo com as suas necessidades. Tanto quanto é do nosso conhecimento, este trabalho representa um esforço pioneiro na incorporação da Privacidade Diferencial na rede Tor. Fornece um mecanismo formal, escalável e eficaz que reforça a privacidade online, resiste a técnicas de censura de última geração e defende o direito fundamental à liberdade de expressão num mundo digital cada vez mais monitorizado.

Palavras-chave: Redes de Anonimato, Privacidade Diferencial, Comunicação Preservadora de Privacidade, Comunicação Encoberta, Ataques de Análise de Tráfego

Contents

List of Figures iv

1 Introduction 1
- 1.1 Motivation . . . 2
- 1.2 Problem . . . 3
- 1.3 Goals . . . 3
  - 1.3.1 Contributions . . . 4
- 1.4 Report Organization . . . 4

2 Related Work 5
- 2.1 Anonymity Networks & Mixnets . . . 5
  - 2.1.1 Groove . . . 6
  - 2.1.2 Loopix . . . 7
  - 2.1.3 Stadium . . . 8
  - 2.1.4 Mirace . . . 8
  - 2.1.5 Problems & Analysis . . . 9
- 2.2 Tor Network . . . 9
  - 2.2.1 Tor Bridges & Pluggable Transport . . . 10
  - 2.2.2 Tor Circuit Scheduling . . . 10
  - 2.2.3 Tor Vulnerabilities . . . 12
- 2.3 K-anonymity . . . 13
- 2.4 Differential Privacy . . . 14
  - 2.4.1 Proporties of Differential Privacy . . . 15
  - 2.4.2 Local Differential Privacy . . . 17
  - 2.4.3 Differential Private Communication . . . 17
- 2.5 Summary . . . 18

3 Elaboration 19
- 3.1 System Model . . . 19
  - 3.1.1 Initial System Model Approach . . . 19

3.1.2 Our System Model Approach 20
3.1.3 Differential Privacy Scheduler 21
3.2 Prototype Guidelines 22
3.2.1 The Tor Approach 22
3.2.2 The MIRACE Approach 22
3.3 Validation 23
3.3.1 Unobservability Measurements 23
3.3.2 Performance Measurements 24
3.4 Summary 24

4 Workplan **26**
4.1 Protopype Development 26
4.2 Tasks & Milestones 28

List of Figures

3.1 Tor Relay Internal Architecture 20
3.2 Enhanced Tor Relay with Differential Privacy Scheduler Component 21

4.1 Gantt chart of the workplan 27

1

Introduction

The internet has become an integral part of the daily life for most of the world's population. However, in some regions, individuals are denied the ability to freely share their stories or access vital information due to oppressive conditions and restrictions on freedom of speech. This oppression and restrictions are often enforced by totalitarian regimes that have vast resources and power to control the flow of information. [0, 0, 0]

As the demand for anonymity networks continues to expand, so does the necessity for robust privacy guarantees to safeguard secure and confidential communications. At the same time, authoritarian regimes intensify their efforts to restrict access exclusively to state-approved content, employing advanced measures to enforce their censorship policies and maintain control over information flow.

In order to enforce and control the network traffic, these regimes resort to the use of traffic analysis techniques to passively monitor the traffic and infer information about the users. Traffic analysis attacks are a significant threat to anonymity networks, as they can be used to infer users' activities and deny their privacy, as well as deanonymize them. [0, 0, 0] Some have demonstrated that advances in machine learning lead to more sophisticated and effective traffic correlation attacks, such as DeepCorr [0] and DeepCoFFEA [0], which leverage deep learning techniques to correlate traffic patterns and infer users' activities with high accuracy, in anonymity networks like Tor. [0]. Statistical Disclosure attacks also pose a threat to anonymity networks, as they identify sender-receiver pairs in anonymity networks. [0, 0] Website Fingerprinting attacks represent a major threat to anonymity networks, as they allow attackers to learn information about user's online activities, even on encrypted networks like Tor, by eavesdropping and collecting side-channel information from the network traffic, between the user and the entry node. With this information, the attacker can use machine learning techniques to infer the user's online activities, such as the websites visited, by analyzing the traffic patterns, with high accuracy. [0, 0, 0]

As these attacks become more sophisticated and effective, journalists, whistleblowers and any other person trying to express their freedom of speech and access information in these regions, are at risk of being identified and persecuted, leading to the decrease of this networks utility, mistrust and fear. Therefore, it is critical that anonymity networks

become more resilient to these attacks and provide stronger privacy guarantees to their users.

1.1 Motivation

Tor, one of the most popular anonymity networks, is an anonymous communication service based on the Onion Routing Protocol. [0] Tor is designed to protect users' privacy and security by routing their traffic through a network of volunteer-operated servers, encrypting it at each step, and ensuring that no single server knows both the source and destination of the traffic.

Anonymity networks remain vulnerable to attacks such as traffic analysis, which threatens the privacy and security of the users these networks are designed to protect. [0, 0, 0, 0, 0, 0, 0, 0] To address this challenge of circumventing censorship, researchers have proposed a range of solutions. Among these solutions, we discuss traffic encapsulation, fully encrypted protocols, and traffic shaping techniques.

Traffic encapsulation is a method that conceals data by embedding packets within other data formats or protocols, thereby obfuscating their true content. This technique leverages protocols, such as those for web browsing and video streaming, to disguise network traffic, making it indistinguishable. [0, 0, 0, 0, 0]

Traffic Shaping is another technique that aims to regulate data transmission to optimize network performance and to prevent timing and correlation attacks, contributing for privacy-prevention. Similarly, traffic shuffling is a technique that focus on reordering packets, with statistical or cryptographic techniques, to make it difficult for an attacker to correlate the traffic. [0, 0, 0, 0, 0]

Researchers have proposed various solutions. Hopper and Vasserman demonstrated that K-Anonymity is effective against traffic analysis attacks, such as Statistical Disclosure and long-term intersection attacks. [0] Nunes et al. proposed BriK, a Tor pluggable transport, that ensures uniform traffic patterns to make users indistinguishable. [0] Some work as Walkie-Talkie [0], Supersequence [0] and Glove [0] also use traffic shaping for defending against website fingerprinting attacks. Wright, Coull, and Monrose proposed Traffic Morphing [0], a traffic shaping technique that uses a Markov Chain to generate traffic patterns that are indistinguishable from the original traffic.

1.2 Problem

Although anonymity networks are designed to provide robust privacy and security, advancements in traffic analysis and correlation techniques have increasingly exposed vulnerabilities, reducing their reliability and trustworthiness for users—particularly in regions where pervasive surveillance by authoritarian regimes is a critical concern. These regimes actively invest in sophisticated surveillance methods and traffic manipulation strategies to maintain control over digital communication.

Tor, despite being one of the most widely used anonymity networks, is not immune to these challenges. It remains vulnerable to several attack vectors, including traffic correlation, congestion-based attacks, timing-based correlations, and fingerprinting techniques. [0, 0, 0, 0] Such vulnerabilities highlight the ongoing arms race between those striving to safeguard online anonymity and those seeking to undermine it through technical and infrastructural advancements.

The absence of standardized privacy metrics for evaluating privacy guarantees in a formal and rigorous manner raises concerns about the effectiveness of anonymizing networks like Tor. Research has demonstrated vulnerabilities in the Tor network to various attacks, including traffic analysis. For instance, Chakravarty et al. showed that statistical correlation techniques can be used to perform successful traffic analysis attacks on the Tor network. Additionally, Shi and Matsuura demonstrated that user privacy can be compromised through fingerprinting attacks.

1.3 Goals

The goal is to add differential privacy (DP) mechanisms into the Tor open-source project to preserve its existing security guarantees while enhancing its privacy protections. By incorporating DP, the aim is to provide quantifiable and adjustable privacy guarantees to users, addressing vulnerabilities to correlation attacks and improving resistance to traffic observability. Furthermore, the project seeks to analyze the trade-offs between privacy and utility within the Tor context and to evaluate the performance of the proposed solution compared to the original Tor network.

As stated in Section, it is important to ensure that anonymity networks are secure against threats like artificial intelligence and machine learning, and to address the lack of standardized privacy metrics to evaluate the performance of anonymity networks. We aim to create and implement a new algorithm that can be used in anonymity networks that leverages differential privacy to ensure privacy and security with higher performance than existing solutions. We also aim to evaluate the performance of the algorithm and compare it with other anonymity networks to prove that this solution is a viable alternative to usual anonymity networks solutions. To test the algorithm and its performance, we intend to realize correlation attacks on the existing networks and on our prototype and compare the results. To test performance, we will measure the latency and throughput of the algorithm and compare it with other anonymity networks.

1.3.1 Contributions

This thesis tries to address the problem of ensuring privacy and security in anonymity networks while maintaining high performance. In particular, the contributions of this thesis are:

1. Design a new Differential Privacy Based Algorithm for Privacy Preserved Communication in Anonymity Networks.
2. Implement the algorithm on top of the Tor network and provide a proof of concept.
3. Evaluate the performance of the algorithm and compare it with other anonymity networks; This contribution aims to prove if this solution is a viable alternative to usual anonymity networks solutions and if, as mention before, can allow users to switch from traditional to anonymity networks.

1.4 Report Organization

The remainder of this report is organized as follows: Chapter introduces anonymity networks and differential privacy, as well as some other related topics. The proposal of work is presented on Chapter, which provides some system model and guidelines for the development and discusses the proof method to verify the work. Finally, workplan is explained and addressed in Chapter

2

Related Work

2.1 Anonymity Networks & Mixnets

Mix networks (Mixnets) were introduced by Chaum [0] as a public key cryptography based protocol designed to obscure the relationship between senders and receivers in a communication network, as well as its content, without the need for a universal trusted authority. Mixnets consist of a series of intermediary servers, called 'mixes', through which messages are decrypted, encrypted, randomly permuted and sent forward. To transmit messages, participants encapsulate their data within successive layers of encryption, with each layer corresponding to a specific mix in the network — a method conceptually akin to onion routing [0]. The length of the encrypted messages is proportional to the number of mixes. Upon receiving a message, each mix removes the outermost layer of encryption, processes the decrypted instructions, and forwards the partially decrypted message to the subsequent mix. This iterative procedure continues until the message arrives at the final mix, which removes the last layer of encryption and transmits the plaintext message to the intended recipient. By design, this system ensures that no single mix possesses complete knowledge of both the sender's identity and the recipient's address, thereby achieving a robust level of anonymity.

Various designs of mixnets have been proposed, based on Chaum work, addressing multiple security and performance issues. Flash Mixing, introduced by Jakobsson [0], focuses on achieving strong anonymity guarantees with reduced latency and computational overhead by shuffling messages, broadcasting the encrypted list to all mixes, who then together compute the output. Hybrid Mixnets [0, 0] efficiently combine public-key and symmetric-key cryptography. Real-time Mixnets [0] aim voice and data communication where continuous data streams have to be transmitted and provide low-latency communication, as long as a certain delay at the start of a connection is tolerable. Work on this field has focused both on security and privacy guarantees, as well as on performance and scalability, with the goal of providing a practical and efficient solution for anonymous communication. Mixmaster [0] allows the sender of a message to remain anonymous to the recipient. Mixminion [0] uses fixed sized messages and supports anonymous replies

and ensures forward anonymity using link encryption between nodes. Onion Routing [0] followed a similar approach, but focused on low latency communication, where messages are encrypted in layers and decrypted by a chain of authorized nodes.

Tor [0] is one of the most popular low latency anonymity system and protects against sender-receiver message linking against a partially global adversary and ensures perfect forward secrecy, messages' integrity, and congestion control. However, Tor is vulnerable to traffic analysis attacks, in case the adversary is able to observe the entry and exit points of the network.

Mix-In-Place (MIP) [0] is a mixnet that uses a cascade of functions in a single proxy, instead of multiple intermediary nodes, to provide anonymity, and proving more resistant to traffic analysis attacks.

Vuvuzela [0] operates in rounds, leading to offline users' inability to receive messages and all messages must transverse a single chain of relay servers. This design protects against both active and passive adversaries unless there is no honest mix node in the network.

2.1.1 Groove

Groove [0] is a scalable, metadata-private messaging system designed to support users with multiple devices, enabling them to send messages at any time, even when recipients are offline, while conserving bandwidth and energy. Built on mixnets, Groove ensures unlinkability between senders and their messages by shuffling batches of messages across servers. Traditional mixnets, however, require all users to submit messages during every round and receive messages at a synchronized rate to prevent correlated traffic patterns, which limits their scalability and usability. To overcome these limitations, Groove introduces a novel approach called oblivious delegation. In this model, users interact with an untrusted service provider that participates in the mixnet on their behalf and synchronizes their clients across all devices. This ensures that even if the service provider is compromised, an adversary cannot infer communication metadata. Unlike prior systems such as Karaoke [0], Stadium [0], and Vuvuzela [0], which require users to be online simultaneously, communicate in synchronized rounds, and support only single-device usage, Groove addresses these restrictions, offering asynchronous messaging and multi-device support. A significant advantage of Groove lies in its ability to minimize the resource demands of each message channel on the mixnet, with a particularly notable reduction in memory usage compared to earlier systems. Groove ensures differential privacy even against attackers with full network control and the ability to compromise multiple servers. Users communicate through persistent message channels, similar to Tor's circuits, while their service provider handles message submission to the mixnet, stores received messages, and synchronizes clients across devices. To maintain anonymity, Groove leverages Parallel Mixnets, which efficiently scale with the number of servers by offering multiple parallel routes for processing messages. Differential privacy is a core goal

of Groove, ensuring that traffic patterns between a user's client and their service provider do not disclose information about the user's communication partners. This is achieved through a scheduler that operates independently of the sender-recipient relationship. In contrast to Loopix [0], Groove maintains user privacy even if their service provider is compromised, eliminating the need for users to operate their own servers.

2.1.2 Loopix

Loopix [0] is a low-latency anonymous communication system offering bi-directional 'third-party' sender and receiver anonymity as well as unobservability. It employs cover traffic and Poisson mixing to provide strong anonymity guarantees and resist traffic analysis by a Global Network Adversary (GNA). The system ensures robust sender-receiver unlinkability against a Global Passive Attacker (GPA) capable of observing all network traffic between users, providers, and mix servers, even in cases where some mix nodes are compromised. Loopix also guarantees sender online unobservability under a corrupt provider and receiver unobservability under the condition of an honest provider. Consequently, the GPA cannot infer the type of transmitted messages, while intermediate nodes remain unable to distinguish between real messages, dropped cover messages, or loops of traffic generated by clients and other nodes. To protect against active attacks, mixes and clients employ self-monitoring mechanisms through self-injected traffic loops. These loops not only act as cover traffic to strengthen anonymity but also enhance sender and receiver unobservability. Similar to Groove, Loopix utilizes service providers to mediate access to the network, manage accounting, and enable offline message reception. These semi-trusted providers enforce rate limits, store messages for offline users, and facilitate message retrieval at a later time. Despite this reliance, Loopix is resilient against adversaries capable of observing all communications and conducting active attacks. A distinguishing feature of Loopix is its continuous operation, unlike Groove's deterministic round-based approach. Messages in Loopix can be retrieved at any time, ensuring users do not lose messages while offline. Furthermore, Loopix employs Poisson mixing, a simplified version of the stop-and-go mixing strategy [0], which introduces independent delays for each message, making packet timings unlinkable. Unlike circuit-based onion routing, where a fixed path is established, Loopix determines the communication path for each individual message independently, even when sent between the same pair of users. This approach enhances privacy and unpredictability. The Poisson mix operates as follows: mix servers listen for incoming packets, check for duplicates, and decode received messages using their private keys. Duplicates are discarded, and the next mix packet is extracted. Decoded packets are not forwarded immediately; instead, each packet is delayed based on a pre-determined delay specified by the source.

2.1.3 Stadium

Stadium [0] is a point-to-point messaging system that ensures metadata and data privacy while efficiently scaling its workload across hundreds of low-cost providers operated by different organizations. Stadium achieves its provable privacy guarantees through the use of noisy cover traffic and differential privacy, which bounds metadata leakage over time. The system employs a mixnet architecture, where user messages and noise messages are distributed among providers and mixed in parallel. Each provider processes only a fraction of the total messages, ensuring scalability and efficiency. As long as at least one provider remains honest, Stadium achieves global verification and secure mixing. Communication in Stadium occurs in fixed rounds, with each round processing a batch of messages accumulated from users during the preceding interval. To enforce differential privacy, servers generate cover traffic at the start of each round. Noise messages are injected into the system in a single, large step before mixing begins. However, this design leaves room for malicious servers to discard noise messages before mixing. To address this vulnerability and ensure that noise remains in the system, Stadium employs cryptographic techniques for verifiable message processing. These techniques enable honest servers to verify the actions of others, preserving the system's differential privacy guarantees. To optimize the computational workload of this verification process, Stadium introduces hybrid verifiable shuffling. Stadium's mixing process relies on a parallel mixing scheme. Each server initially processes a small fraction of input messages, mixes them, and redistributes the mixed messages among other servers. This cycle of mixing and redistribution is repeated multiple times. Although messages are initially partitioned based on their originating server, the repeated cycles of splitting and mixing result in global mixing across all servers.

2.1.4 Mirace

MIRACE [0] is a censorship circumvention system developed by Pereira that leverages dynamically construsted circuits to provide secure communications. This work allows traffic to be split across multiple circuits, each composes of several nodes. This solutions combines traffic splitting with encapsulations, throught the combination of TLS, QUIC and WebRTC tunnels for diverse covert communitation strategies, and trafic shapping, by the addition of jitter and padding. Pereira demostrated that MIRACE is able to ensure secure communications against correlation attacks, even with machine learning-based website fingerprinting attacks. The author also showed that MIRACE is able to maintain practical levels of latency and throughput, even with the addition of nodes. Under fingerprinting attacks, MIRACE showed strong levels of resistance with high accuracy and the ability to effectively obscure traffic patterns.

2.1.5 Problems & Analysis

Mixnets have been analyzed in terms of security and privacy guarantees, as well as performance and scalability. Zhu et al. revealed that flow-correlation attacks can be used to de-anonymize users in mixnets, in relation to the system parameterization, such as sample size, noise level, payload flow rate, and detection rate. [0]. Shmatikov and Wang demonstrated that mix networks are vulnerable to timing analysis, even with defenses in place, by the use of advanced statistical techniques, such as cross-correlation and machine learning, successfully de-anonymizing users. [0].

2.2 Tor Network

Tor is a circuit-based low-latency anonymous communication service [0] based on *onion routing* that aims to anonymize TCP-based applications, such as web browsing, SSH and instant messaging. The network relies on *Onion Routers* (ORs) that are responsible for maintaining TLS connections to every other OR and for forwarding traffic along the circuits, and are also maintained by volunteers.

An *Onion Proxy* (OP) is the local software run on the client and that handles connections with the users' applications, fetches the current network information and the lists of *Onion Routers* (ORs) and builds circuits through the network. Each OP maintains TLS connections to nodes they have been in contact recently and a pair of keys: long-term identity key and short-term onion key. Long-term keys are used to sign TLS certificates and its router descriptor. Short-term keys are used to decrypt requests, negotiate ephemeral keys and set up circuits.

Traffic travels the network through 512 bytes fixed-size *cells* with a header and a payload. The header contains the circuit identifier and the command to describe what to do with the payload. Based on the header's command, cells are either control cells or relay cells. Control cells are used to manage circuits and connections, while relay cells are used to carry data and can only be sent after a circuit is established. Relay cells have a digest assigned by the sender and its header and payload are iteratively encrypted with the symmetric key of each hop up to the target OR. This way, only the last OR in the circuit can read the payload, given the fact that the digest is encrypted to a different value at each step.

The sequences of ORs which traffic is routed through are called *circuits*, normally composed by 3 relays (entry or guard, middle and exit). To create a new circuit, a user's OP must execute a Diffie-Hellman key exchange with each OR in the circuit, one hop at a time. This way, each OR knows the previous and the next node in the circuit, but not the source and destination of the data. Once the circuit is established, relay cells are sent through the circuit,

2.2.1 Tor Bridges & Pluggable Transport

One vulnerability about Tor is the traffic blocking by ISPs of censored regimes. One way to perform this blocking is by blacklisting Tor relays, since the list of their IP addresses is public, which prevents clients from establishing circuits. Bridges come as a solution to this problem, where unpublished proxies foward client's traffic to an Tor entry relay, making the client connect to some unlisted bridge, rather than some potentially blacklisted relay [0]. Nevertheless, bridges are still vulnerable to traffic fingerprinting attacks. Matic, Troncoso, and Caballero demonstrated that 55% of public bridges are vulnerable to the traffic blocking referred before. Even with encrypted traffic, Tor's traffic is still indentifiable by its packet size, currently fixed-sized 512 byte cells, and by the TLS byte patterns, more precisely the TLS extension called Server Name Indication (SNI), which adds the domain name to the TLS header without any encryption.

To overcome this issue, Tor bridges support *pluggable transports* (PT) [0] which transform the Tor traffic flow between the client and the bridge, making traffic monitoring between these look innocent, instead of actual Tor traffic. Some examples of PTs are *obfs4*, *meek*, *Snowflake* and *WebTunnel* [0]. *obfs4* obfuscates Tor traffic to make it appear random, thwarting efforts by censors to detect it through Internet scanning; *meek* disguises Tor traffic to resemble ordinary browsing activity on prominent websites; *Snowflake* routes your connection through volunteer-operated proxies to make it look like you're placing a video call instead of using Tor; and *WebTunnel* camouflages Tor traffic by making it indistinguishable from standard HTTPS requests, thereby creating the illusion of accessing a secure website.

2.2.2 Tor Circuit Scheduling

Traffic handling in Tor involves several buffers and schedulers. Each circuit is used by only one client but if multiple circuits use the same two ORs, they share the same connection. That is, OR will have multiple simultaneously connections with other ORs but only one to any given OR, and each connection will transport data for various circuits. When a OR receives a packet from an TCP stream, the packet gets demultiplexed, together with other possible incoming packets from different TCP streams, and they get placed into the kernel socket input buffers. Here packets are processed by the OS, usually in FIFO order, and sent to the Tor input buffers. Upon receiving the packets, Tor will remove the TLS layer and onion-encrypted (or onion-decripted, depending on the circuit's direction) and finally enqueued in the Tor Circuit Queue. Each relay maintains a queue for each circuit that it serves. Cells from the same Tor input buffer might not be enqueued in the same circuit queue, as they might belong to different circuits. The cells are then selected, dequeued, onion-encrypted and stored in a Tor output buffer. The data is then written to a kernel socket output buffer when the Tor output buffer contains sufficient data to form a TLS packet.

Tor has a congestion control mechanism (Circuit-Level Throttling), designed to regulate

communications across circuits, which ensures resource distribution, prevents network congestion and provides safety against attacks such as Denial-of-Service. As Tor is a low latency network and its security guarantees rely on how many users are using the network, it is important to have a good congestion control mechanism and practical levels of latency and throughput. However, it has been shown that Tor's Circuit Scheduling Algorithm allows busy circuits to crowd out bursty circuits, leading to congestion and latency issues [0]. Additionally, Jansen et al. have demonstrated that Tor's congestion occurs in the kernel socket buffers. Both these works have proposed solutions to improve Tor's congestion control and latency issues.

Even with the improvements in congestion control, as merging Tang and Goldberg and Jansen et al. works, Tor is still vulnerable to traffic analysis attacks such as website fingerprinting and correlation-based attacks [0]. Even though none of these works presented additional security risks or vulnerabilities to the Tor network, they have shown little or no improvement in the network against these attacks.

2.2.2.1 Exponential Weighted Moving Average (EWMA)

Tang and Goldberg proposed a circuit scheduling algorithm that handles circuits by their recent activity, where bursty circuits have higher priority over busy ones [0]. Generally, bursty circuits are those used for web browsing and instant messaging, and also referred as interactive streams. On the other hand, busy circuits are those used for file transfers, and also referred as non-interactive streams. Interactive streams are more sensitive to latency while non-interactive streams tolerate higher delays. The approach is based on the Exponential Weighted Moving Average (EWMA) algorithm, which assigns a weight to each circuit based on the number of cells sent on each circuit. When selecting the circuit to process, the algorithm chooses the one with lowest EWMA value. Usually, newly created circuits have a lower EWMA value, leading to a higher priority. Even though this algorithm has merged into Tor, it has been demonstrated that it actually reduces performance for clients under some network conditions [0]. Additionally, this work does not address the adversaries' ability to correlate and fingerprint users' traffic, which is a major issue in Tor network.

2.2.2.2 Kernel Informed Socket Transport (KIST)

Motivated by the still congestion problem with Tor and the lack of understanding of where this problem occurs, Jansen et al. found that the congestions occurs inside the kernel socket buffers and on Tor's sockets management. They proposed a new scheduling algorithm, Kernel Informed Socket Transport (KIST), that solves these problems by choosing from all circuits with writable data rather than just those belonging to a single TCP socket and by dynamically managing the amount of data written to each socket based on real-time kernel and TCP state information that can be queried from user space. This way, KIST reduces Tor's circuit congestion by more than 30%, reduces network latency by 18%, and increases

network throughput by nearly 10% [0]. KIST was merged and configured as default socket scheduling algorithm in Tor since January 2018, replacing the EWMA-based algorithm, referred in Section 2.2.2.1. Although, KIST authors acknowledge that it does not affect the adversaries abilities to collect accurate measurements required for the throughput correlation attack, compared to the 'vanilla' Tor scheduling algorithm.

2.2.3 Tor Vulnerabilities

Studies have pointed that Tor is susceptible to traffic analysis attacks, where an adversary can infer information about a user, like who is communicating with whom, by observing the traffic patterns. Here we cover some of the most common traffic analysis attacks against Tor, which have been demonstrated to be effective in de-anonymizing users.

2.2.3.1 Website Fingerprinting Attack

Chakravarty et al. also proposed a traffic analysis attack model against Tor where the adversary uses statistical correlation over the server to exit and entry to client traffic to find similar traffic patterns [0]. This attack consists on clients generating sustained traffic for a long period of time, by downloading a file for example, and the adversary being therefore able to inject traffic patterns by perturbing the TCP connection. This way, the adversary can obtain traffic analysis from the server to exit node and correlate it with the traffic from the entry node to the client, by finding the flow which carries the injected fingerprint. Testing showed that the attack was able to correctly identify the source of anonymous traffic 81.4% of the time. Deep Fingerprinting [0] is a website fingerprinting designed using deep learning techniques, that uses a simple input format and does not require handcrafting feature for classification. The authors demonstrated that this attack is 98.3% accurate in identifying the website visited by the user. Sirinam et al. proposed a website fingerprinting attack that uses triplet networks and a machine learning technique, that requires few training samples, called N-shot learning, which reduces the effort of gathering and training with large datasets [0]. Triplet networks. Finally, this work achieved up to 95% accuracy in identifying the website visited by the user, only by using 20 examples per website.

2.2.3.2 Correlation-Based Attacks

Correlation attacks were acknowledged by Dingledine, Mathewson, and Syverson [0] as a potential threat to Tor's anonymity. These attacks exploit the adversary's ability to observe both the entry and exit points of the network, allowing them to correlate the traffic patterns and de-anonymize users. Sun et al. [0] proposed asymmetric traffic analysis as an end-to-end timing analysis that allows AS-level adversaries to compromise Tor users' anonymity. Internet path from exit relays to the web server may differ from the path from the web server to the exit relay, allowing the adversary to observe the TCP

acknowledgement traffic on the path from the server to the exit relay, even if the adversary is enabled to observe the data traffic on path from the exit relay to the server. This attacks might be applicable where the adversary observes: data traffic from the client to the entry relay and from the exit relay and the server; data traffic from the client to the entry relay and TCP acknowledgement traffic from the server to the exit relay; TCP acknowledgement traffic from the entry relay to the client and data traffic from exit relay to the server; or TCP acknowledgement traffic from the entry relay to the client and TCP acknowledgement traffic from the server to the exit relay. The authors proposed a suite of new attacks against Tor called 'RAPTOR', where they demonstrated an accuracy of 95% in correlating client/server pair.

DeepCorr [0] and its extension DeepCoFFEA [0] are both systems that leverage advanced deep learning techniques to correlate traffic. DeepCorr is able to correlate Tor flows by first using deep learning to learn a correlation function and then using this function to cross-correlate the traffic. In contrast to website fingerprinting, this work has no need to learn target destinations, instead the function can be used to link flows on arbitrary destinations. Additionally, it does not require the traffic to be sent in the same circuits used to learn the function. DeepCoFFEA extends DeepCorr by using a modified truplet network approach and ampplification techniques, demonstrating that it achieved lower computational costs and higher efficiency compared to DeepCorr, improving the accuracy of the correlation attack.

2.3 K-anonymity

To address the exponential growth in the number and variety of data collections containing person-specific information, and the pressing need to prevent privacy compromise, *k-anonymity* [0] emerged as a foundational privacy model. This model ensures data anonymity not only by removing explicit identifiers such as names, addresses, or phone numbers but also by protecting against re-identification through linking or matching data with external sources. It achieves this by mitigating the risk posed by unique characteristics within the dataset.

Earlier work by Sweeney demonstrated the vulnerability of anonymized data to linkage attacks using the 1990 U.S. Census. They highlighted that 87% of the U.S. population reported characteristics that rendered them unique, and over half of the population could be uniquely identified using just three attributes: place, gender, and date of birth [0]. To counter these risks, Sweeney defined *k-anonymity* as a property satisfied by a dataset `D` if, and only if, each sequence of values for quasi-identifiers in `D` is indistinguishable from at least $k-1$ other records in `D`. In practical terms, any tuple in `D` must be identical to at least $k-1$ other tuples concerning its quasi-identifiers. This property ensures that the data cannot be easily linked to other sources and protects users from re-identification, with the privacy guarantees strengthening as k increases.

Ahn, Bortz, and Hopper extended the concept of *k-anonymity* to communication systems, defining a protocol as sender k-anonymous if it ensures that an adversary attempting to identify the sender of a message can narrow their search to a set of k potential senders. Similarly, receiver k-anonymity guarantees that an adversary can only narrow down the possible recipients to a group of k [0].

Despite its foundational role, *k-anonymity* remains vulnerable to several types of attacks, including unsorted matching, complementary release, and temporal attacks, as noted by Sweeney [0]. Unsorted matching attacks exploit the order of tuples in a released dataset, which can be mitigated by randomizing the tuple order. Complementary release attacks arise when subsequent data releases are combined, allowing adversaries to re-identify individuals by correlating datasets. Temporal attacks exploit data changes over time, such as additions, deletions, or modifications, leading to violations of k-anonymity guarantees as datasets evolve.

Further research has explored enhancing k-anonymity to address its limitations. For instance, Hopper and Vasserman observed that receiver k-anonymity improves effectiveness against long-term intersection and statistical disclosure attacks. Additionally, incorporating periodic sender k-anonymity under low churn conditions enhances resistance to mass surveillance, albeit without achieving an optimal cost function [0].

Recognizing that k-anonymity alone may not suffice to protect privacy in the era of big data, Gosain and Chugh proposed combining k-anonymity with Differential Privacy. This hybrid approach addresses the challenges posed by the massive scale and interconnected nature of modern data systems [0]. Similarly, in response to the proliferation of social networks and the accompanying surge in data collection, Campan and Truta introduced an anonymization technique for social network data, masking it according to the k-anonymity model [0].

While k-anonymity offers practical privacy guarantees, it is primarily suited for systems prioritizing efficiency over robust privacy protections. It remains less applicable in scenarios where strong, comprehensive privacy guarantees are essential.

2.4 Differential Privacy

Differential Privacy [0, 0, 0], introduced by Dwork et al., is a formal notion of privacy and is a property of algorithms, rather than data like k-anonymity. A algorithm or function satifies differential privacy if for all neighboring datasets x and x', and all possible sets of outputs S:

$$\frac{\Pr[F(x) \in S]}{\Pr[F(x') \in S]} \leq e^{\varepsilon} \tag{2.1}$$

In this privacy mechanism definition, F is a randomized funtion, and its output will be similiar, with or without the data of any specific individual. Moreover, F's randomness should be enough so that outputs do not reveal the input data. This way, we can ensure

that the privacy of any individual is preserved, by ensuring plausible deniability. This happens because the output of the function cannot be correlated to any specific input, whether the input is present or not. Furthermore, the mechanism can maintain certain level of accuracy, due to having a precise understanding of the noise generation process. Differential Privacy intends to reject no malicious adversary, instead it ensures that the adversary is not capable of inferring any specific information, due to the randomness of the output.

Another important requirement of differential private mechanisms is the ϵ paramenter [0, 0, 0, 0]. This paramenter is called *privacy parameter* and it controlos the 'amount of privacy' that the correspondent function provides. Small values of ϵ require F to provide very similar output when given similar inputs, originating in higher levels of privacy. On the other hand, higher values of ϵ allow outputs to diverge more, leading to lower levels of privacy. In practice, ϵ should be less or equal to 1, and no greater than 10, to provide meaningful privacy guarantees.

Differential Privacy originated from the challenge of enabling the extraction of meaningful insights about an underlying population while rigorously safeguarding individual privacy. Therefore, the earlier definitions of this concept were focused on datasets and queries over them.

The Laplace Mechanism was proposed together with the definition of Differential Privacy by Dwork et al. [0]. The simpliest way to achieve differential privacy is by adding noice to the output of a function. The challenge is to balance the injected noise in order to have enough to satisfy the definition of DP but not too much to make the output useless. The Laplace Mechanism is a simple mechanism which, for a function $f(x)$ that returns a real number, $F(x)$ satifies ϵ-differential privacy:

$$F(x) = f(x) + \text{Lap}(\frac{s}{\epsilon}) \tag{2.2}$$

where s is the sensitivity of the function f, and $Lap(S)$ denotes sampling from the Laplace distribution with scale S and center 0.

The sensitivity of a function is a measure of how much the output of the function can change when the input changes. This property is important to determine how much noise should be added to the output of the function, in order to guarantee differential privacy.

2.4.1 Proporties of Differential Privacy

In this section, we cover the three main properties of Differential Privacy: Sequential Composition, Parallel Composition and Post-Processing.

2.4.1.1 Sequential Composition

Sequential Compositions [0, 0] is a major property of Differential Privacy, which determines the total privacy cost of releasing multiple results of differential private mechanisms on

the same input. Formally, this property states that if $F_1(x)$ satisfies ϵ-differential privacy and $F_2(x)$ satisfies ϵ'-differential privacy, then the composition of these two functions, $G(x) = (F_1(x), F_2(x))$, satisfies $(\epsilon + \epsilon')$-differential privacy. This property is important to algorithms that access data more than once. The bound on privacy cost given by this property is an upper bound, and the actual privacy cost can be lower than the sum of the individual privacy cost.

2.4.1.2 Parallel Composition

Parallel Composition [0, 0] can be considered as an alternative to Sequential Composition, as it determines the total privacy cost of releasing multiple results of differential private mechanisms. The idea consists on splitting a dataset into disjoint chunks and applying a differentially private mechanism to each chunk separatly. Formally, if $F(x)$ satisfies ϵ-differential privacy, and we split the dataset X into k disjoint chunks such that $X = X_1 \cup X_2 \cup \ldots \cup X_k$, then the mechanism that releases all of the results $F(x_1), \ldots F(x_k)$ satisfies ϵ-differential privacy. Comparatively to Sequential Composition, Parallel Composition gives a better upper bound. Since F is run k times, Sequential Composition states that this procedure satisfies $k\epsilon$-differential privacy.

An issue with Differential Privacy arises with small datasets. A large dataset is able to achieve a strong privacy guarantee with relatively weak noise, and allows the results to be useful. However, small datasets require stronger noise to achieve the same privacy guarantee. As we split the dataset into chunks, we must care for the utility of the results and the impact of the number of chunks that we split the dataset into. The more chunks we split the dataset into, the stronger the noise we must add to the results, and the less useful the results will be.

2.4.1.3 Post-Processing

The last property is Post-Processing [0, 0] and it states that is impossible to reverse the privacy protection provided by a differential privacy by post-processing the data in some way. Formally, if $F(x)$ satisfies ϵ-differential privacy, then for any function h, $h(F(X))$ satisfies ϵ-differential privacy. This means that no danger will arise from performing additional computations on a output of a differential private mechanism, as the privacy guarantee will be preserved (and not reversed). This property is essetial to ensure that differential privacy is resitant against privacy attacks based on auxiliary information and that attacks effectiveness is only limited by the privacy parameter ϵ.

2.4.2 Local Differential Privacy

Differential Privacy was initially proposed in a central model, where sensitive data was collected in a single dataset and the data curator must be a trusted entity in order to corretly execute differential private mechanisms, even in scenarios where the analyst is

malicious.[0] However, this assumption might not be very realistic. In practice, the data curator and the analyst might be the same entity, and the data curator might not be trusted, leading to no differential private mechanisms being in place. This way, *Local Differential Privacy* (LDP) raises as an solution in which data is made differential privacy before being collected by the data curator, being already used by big companies such as Google [0] and Apple [0].

Randomized Response is a mechanism for LDP, proposed by Warner [0] in 1965, intended to improve survey responses about sensitive issues, but was not originaly design as a mechanism for Differential Privacy. Dwork and Roth presented a variant of this mechanism, in which the subject flips a coin to answer a 'yes' or 'no' question. If the coin is heads, the subject answers truthfuly, otherwise, the subject flips the coin again and answers 'yes' if heads and 'no' if tails. The randomization in this algorithm comes from the two coin flips, creating uncertainty about the true answer, and therefore providing privacy.

2.4.3 Differential Private Communication

Some studies have been conducted on applying Differential Privacy to communication systems [0, 0, 0]. These studies focus on providing privacy guarantees to users and protect them agains traffic analysis attacks, powered by machine learning techniques.

Zhang et al.[0] proposed a differential private mechanism for streaming data, by the use of a Chrome extension with a DP mechanism that proxies streams between the browser and the server. The extension intercepts the requests from the client, which sends requests on behalf of the client based on a differntially private mechanism, instead of instantly relaying them immediately. The authors found that differntial privacy lowered the machine learning techniques, used by the adversary, effeciency, even though the accuracy of the attack was less affected in case of the adversary training model with the same DP mechanism.

NetShaper [0] is a network side-channel mitigation system, proposed by Sabzi et al., based on traffic shapping, which provides quantifiable and tunable privacy guarantees, through the use of differential privacy. This work shapes traffic based on a DP-mechanism that adds noise to the traffic, that can be split into 3 steps: *queue, query, and post-process*. In the first step, the system queues the input traffic. After a fixed parameterizable periodic interval, the system queries the queue and adds noise to the traffic, perfomed by the differential private traffic shapping algorithm. Finally, the packets are post-processed and transmitted to the network.

Vilalonga et al.[0] suggested Randomized Response could be used to strengthen the privacy of the Tor network, by adding noise to the traffic, making it harder for adversaries to correlate the traffic and de-anonymize users. The authors proposed a mechanism that uses pluggable transports to connect the client to the Tor network and a noise-adding algorithm. This algorithm sends packets based on the Randomized Response mechanism, in this case, where the algorithm chooses to send a packet or a noise packet based on a

probability.

2.5 Summary

Anonymity Networks have been around for almost 40 years but advances in machine learning and traffic analysis techniques have compromised the privacy guarantees that these networks aim to provide. Tor is a widely used low-latency anonymity network that has been shown to be vulnerable to such attacks. We take a particular interest in the developments on Tor's scheduling algorithms that proposed (and later been merged into the source code) solutions to improve Tor's congestion control. The works referred in this chapter have both demonstrated effects in performance and congestion control but lack on providing privacy guarantees that Tor aims to provide. Some work on anonymity network, including Tor, have tryed to mitigate these attacks, such as website fingerprinting and correlation-based attacks, but they lack on giving formal privacy guarantees. In most of the cases, privacy and security are pmçy measered by pratical testing, which may not be enough to ensure that the network is secure and private, if some conditions are not rigorously tested and proven. To address this problem, this dissertation aims to provide formal privacy guarantees to anonymity networks, more precisely to Tor, by implementing a Differential Private Tor Socket Scheduler that, which also offers adaptable and parameterizable privacy protections. To this day, there is no work that focus on appliying Differential Privacy techniques into Tor network.

3

Elaboration

In this chapter, we give a conceptual overview of the proposed solution. We begin by presenting the general overview of the system model and the design goals in Section 3.1, followed by a more technical description of the techniques and technologies used in the implementation of the prototype in Section 3.2. Finally, we present how we intend to test and validate our implementation in Section 3.3.

3.1 System Model

In this section, we present the system model of our proposed solution. We start bu presenting the foudation of our work in Section 3.1.1, where we explore the current Tor Relay internal architecture and how we intend to extend its architecture. In Section 3.1.2, we present our approach to enhance the privacy of Tor users by integrating a differential privacy mechanisms in Tor relays and on the Client side. Finally, in Section 3.1.3, we present the Differential Privacy Scheduler, or DP Scheduler, which is a component that maintains the circuit queues and is responsible for selecting the cells to be allocated in the output buffer, just like the vanilla Tor scheduler. Next, we present the Prototype Guidelines in Section 3.2, where we present the two main approaches to implement our solution: the Tor Approach and the MIRACE Approach, and a brief explanation of how we intend to implement them. Finally, in Section 3.3, we present how we intend to validate our work.

3.1.1 Initial System Model Approach

As shown in Section 1.2, Tor is a widely used network for anonymous communication. However, it has some vulnerabilities that can be exploited by attackers to de-anonymize users, such as traffic analysis attacks. Several techniques have been proposed to mitigate these attacks, such as k-anonymity, but they lack formal guarantees of privacy and are almost based on practical validations, which in real-world applications might vary. In this context, we proposed a new approach to enhance the privacy of Tor users by integrating a differential privacy mechanisms in Tor relays and on the Client side. Our approach aims

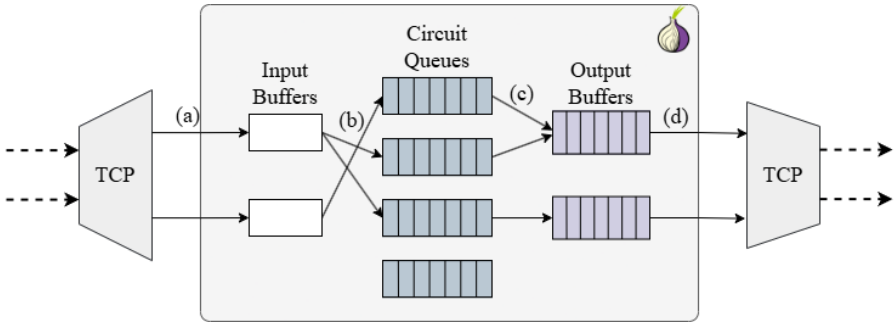

Figure 3.1: Tor Relay Internal Architecture

to develop a Differential Private Scheduler similar to KIST (Section 2.2.2.2) that will be responsible for adding noise to the traffic flows in order to protect the privacy of users. The Tor scheduler handles two buffers, input and output. The first receives the cells and the second sends them. Between these two buffers, the circuit queue is responsible for managing the cells that are being processed.

The Tor Relay Internal Architecture is shown in Figure 3.1. When a relay receives a TCP packet, the kernel processes them and delivers to Tor (Figure 3.1 (a)). Upon receiving the entire packet, Tor onion-encrypts it and the cell is enqueued into the circuit queue (Figure 3.1 (b)). Each circuit queue represents a circuit that the relay maintains. In this example, this relay is part of 4 circuits, each with a different circuit queue. Nonetheless, this does not mean that the relay must have an input buffer for each circuit, since a single TCP connection may be part of multiple circuits. The scheduler is responsible for selecting a cell from the circuit queue to be sent to the output buffer (Figure 3.1 (c)). Finally, when the relay's output buffer contains sufficient data to form a TLS packet, the data is sent via TCP connection (Figure 3.1 (d)) [0, 0].

3.1.2 Our System Model Approach

The Tor Relay described in Figure 3.1 represents a simple relay. Our goal is to provide formal guarantees of privacy to users of Tor network whilst maintaining practical levels of performance. To achieve this goal, we propose a new approach to enhance the privacy of Tor users by integrating a differential privacy mechanisms in Tor relays and on the Client side. Our approach aims to develop a differential private mechanism, called *Differential Privacy Scheduler Component*, that is responsible for adding noise to the traffic flows based on several parameters in order to protect the privacy of users. Figure 3.2 illustrates the enhanced Tor Relay internal architecture with our Unobservability Scheduler Component. This component is responsible for adding noise and control to the traffic flow, focusing on improving the circuit queue scheduler algorithm. This component has a variable levels of performance and privacy, according to the parameters set by the relay or dynamically calculated.

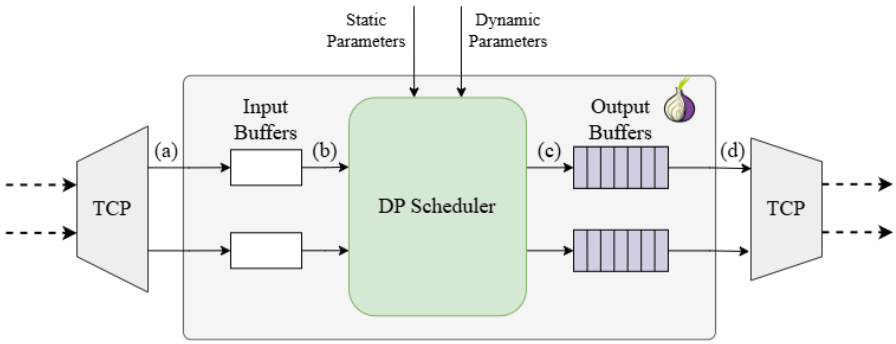

Figure 3.2: Enhanced Tor Relay with Differential Privacy Scheduler Component

3.1.3 Differential Privacy Scheduler

The Differential Privacy Scheduler, or DP Scheduler, is a component that maintains the circuit queues and is responsible for selecting the cells to be allocated in the output buffer, just like the vanilla Tor scheduler. In addition, the DP scheduler uses a set of parameters to allow customization on the level of privacy and performance. These parameters can be split into two categories: configuration and dynamic. This component represents a subtle change in the Tor Relay internal architecture, but we believe that it can significantly improve users' privacy, based on the differential privacy formal guarantees that we aim to provide.

3.1.3.1 Configuration Parameters

The Configuration, or Static, Parameters parameters are set by the relay operator. One of these parameters is the queue size, called privacy window, which can have two different approaches: the *privacy window length* determines the number of cells that the DP Scheduler will consider executing the differential private algorithm and, therefore, apply noise into this window and subsequently into the traffic; the other approach, called *privacy window time interval* determines the time, in miliseconds, to fill a buffer and, when the timer ends, the differential private mechanisms is applyed on the received packets. We will consider implementing both approachs in our prototype. Another static parameter is the privacy parameter, often denoted by ϵ, which determines the level of privacy that the DP Scheduler will provide. Our work aims to provide a configurable file that allows the relay operator to set these parameters before running the relay.

3.1.3.2 Dynamic Parameters

These parameters are calculated by the DP Scheduler based on the current state of the relay and the network. An example of dynamic parameter is the allowed interval of latency that our system must aim to provide. In this case, the DP Scheduler will calculate the noise to be added to the traffic in order to maintain the latency within the allowed interval.

If the latency is above the interval, the value of the static parameters ϵ may be lowered, reducing the noise injection and, therefore, improving the performance.

3.2 Prototype Guidelines

In order to implement the proposed solution, we have 2 main approaches: the Tor [0] approach and the MIRACE [0] approach. The Tor approach consists of modifying the Tor source code to include the Differential Privacy Scheduler Component. The MIRACE approach consists of creating a new relay that will be responsible for adding noise to the traffic flows.

3.2.1 The Tor Approach

Firstly, we will consider the approach based on extending the Tor Relay Scheduler Architecture with the implementation of a differential private mechanism capable of adding sufficient noise to the traffic flows that is able to reduce website fingerprinting and correlation attacks on the Tor network, therefore protecting the privacy of users. To implement this approach, we will use the already existing Tor source code, more specifically the scheduler component (`scheduler.h`). This approach aims to modify the `scheduler.h` file and add our differential private scheduler as part of the `scheduler_type_t` enum, which already contains the vanilla and the KIST [0] schedulers. In addition, we aim also to develop a new scheduler, using the Tor Scheduler Interface `scheduler_t`, which implements the necessary functions in order to fit our design into Tor system. This way, the file 'scheduler_dp.c' would be added to the Tor source code. The source code is written in C language and, even though there has been some efforts to export the C code base [0] to Rust code base [0], after analyzing both sources, we decided to only implement in the C language, considering that, at the time of our work, the Rust source code only represents a fraction of the Tor system, and we believe that it could affect the validation of our work. This approach aims to improve Tor system by providing greater guarantees and formal proof privacy-wise, and we aim to maintain reasonable performance results in order to the scheduler to be considered as an option for Tor users which may consider sacrificing some performance in order to have a better privacy.

3.2.2 The MIRACE Approach

On the other hand, we also propose another approach for the implementation of our Differential Private Component, by extending the MIRACE work. This way, we believe that merging both path splitting and differential privacy mechanisms can provide a better privacy. This work is based on another NOVA LINCS work called TorKameleon [0], which is developed in Go language. The MIRACE approach consists of implementing the Unobservability Scheduler Compenent in each Relay Node used in the MIRACE network. Additionally, we aim to augment the scheduler with a formal and differential private

mechanisms to levarage the MIRACE multi-path algorithm. We believe that combining traffic splitting and multipath multi-encapsulation techniques from MIRACE with the differential privacy mechanisms from our work can provide a better privacy to Tor users. This approach is the alternative to the Tor approach, considering that Tor source code may be more complex and harder to implement in the time frame of this work. On the other hand, the MIRACE approach uses Golang which is a simpler language and both MIRACE and TorKameleon authors represent a great support to our work.

3.3 Validation

The main goal of this work is to provide a viable solution to the current privacy issues in Anonymity Networks, such as Tor, by using Differntial Privacy mechanisms to provide formal privacy guarantees whilst maintaining reasonable performance levels. Using DP algorithms, we aim to provide a solution that can decrease the efficiency of orrelation attacks and website fingerprinting. In order to validate our solution and present a trustworthy conclusion of wether our work is useful to the current Anonymity Networks State-of-the-Art or it can be ruled out for future work and reference, we will present two main types of validation: Performance Measurements and Unobservability Measurements. To execute our validation, we will use Shadow [0, 0], which is a specific experimnetation tool that allows us to test and simulate the Tor network in a simulated environment, leading to a more controlled but easier prototype implementation and validation.

3.3.1 Unobservability Measurements

Unobservability is the main goal of our work. We aim to prove that Differential Privacy mechanisms can provide strong privacy guarantees and that its a real-world solution Even though performance is a critical measurement, we believe that the privacy guarantees are the most important aspect of our solution. To do so, we will consider Website Fingerprinting as the main attacker technique over our system. To provide strong evidence of the privacy guarantees of our solution, we will consider the following machine-learning models for website fingerprinting:

- DeppCoFFEA
- TorMarker
- TirDeanon
- TNR

In addition, we want to formally prove the differential private mechanisms that we aim to provide

3.3.2 Performance Measurements

Even though the main goal of our work focus on privacy and the effectiveness of Differential Privacy, we consider performance to be the key to the success of our solution, which can be the difference between a viable solution, useful in real-world applications, and a theoretical solution which works but does not provide a real-world application due to low performance. This way, the Performance Measurements aim to provide a extensive and detailed analysis of the performance of our solution. In order to achieve this, we will consider various metrics, such as latency and throughput. We will also consider different scenarios where we can stress our solution in different ways, leading to a more detailed and broad analysis of the performance of our solution. We will have 5 main scenarios:

- **iPerf:** This tool will be used to measure some critical metrics, such as latency and throughput, but also the jitter and packet loss that our solution may introduce.
- **Web Browsing:** In this scenario, we consider that it is important to evaluate perfomnace measurements over a scenario where high performance is required, such as web browsing. This scenario will have a high number of small sized requests and responses.
- **File transfers:** This scenario will simulate the transfer of large files over the network to evaluate throughput and latency under sustained, high-bandwidth usage. It will help identify how the scheduler handles prolonged data flows and ensures privacy without significantly impacting performance.
- **Video Conference Meeting:** This scenario will test real-time communication with strict latency and jitter requirements. It will assess how the scheduler performs under conditions where low delay is crucial for maintaining audio and video quality.
- **Streaming:** Streaming scenarios will evaluate continuous data delivery performance, focusing on how well the scheduler maintains a balance between bandwidth usage and differential privacy guarantees, especially under varying network conditions and sustained traffic.

With these scenarios and metrics, we believe that we can provide a comprehensive analysis of the performance of our solution, which will help us to understand the trade-offs between privacy and performance. The aim is to demosntrate that our solution can provide a reasonable level of privacy without significantly impacting the performance.

3.4 Summary

We propose to implement a Unobservability Scheduler Component, based on the Tor Scheduler and considering some implementations over it, like KIST and EWMA, but aiming to provide greater and better privacy guarantees to users. By providing a formal

and differential private mechanism over Relay Nodes, we intend to protect the privacy of users, whilst maintaining feasable performance levels. We will consider two main approaches from the start: the Tor Approach and the MIRACE Approach. The Tor Approach consists of extending the Tor Scheduler with our Differential Privacy Scheduler Component, whilst the MIRACE Approach consists of extending the MIRACE Relay Nodes with our Differential Privacy Scheduler Component. To validate the solution, we will consider two types of measurements: Performance and Unobservability. The Performance Measurements aim to provide a comprehensive analysis of the performance of our solution, considering different scenarios and metrics, such as latency and throughput. The Unobservability Measurements aim to provide a comprehensive analysis of the privacy guarantees of our solution, considering different scenarios and metrics, such as website fingerprinting and correlation attacks. We believe that by providing a comprehensive analysis of the performance and privacy guarantees of our solution, we can demonstrate that our solution can provide a reasonable level of privacy without significantly impacting the performance.

4

Workplan

To provide a clear overwiew of thr workplan, we create a Gantt chart, displayed in Figure 4.1, and a table with the tasks and their respective description and tasks dependency. Our workplan is desing for an estimated effort similar to the compleplated work for a Master Thesis, corresponding to 30 ECTS, meaning that we plan to spend 840 hours of work (30 ECTS * 28 hours/ECTS), for which we account for the possibility of a margin of error of 10% in the total time.

4.1 Protopype Development

To mitigate the risk of not being able to complete the work in the estimated time, we divided our solution into phases:

- **Phase 1:** Firsly, we will implement the prototype of the *Unobservability Scheduler* with basic functionalities and a randomized response-based mechanisms. In this phase, we prosume to spend a reasonable amount of time setting up and understading Tor's or MIRACE's codebase, depending on the approach we take. With this basic differntial private scheduler, we will already be able to test the basic functionalities of the system, and we will be able to evaluate the performance of the system in terms of privacy and utility. We also want to provide detailed evidence of the impact of each parameter. This way, this prototype will work as reference of the system's performance and will be used to compare with the final version of the system. Therefore, we will have to develop our benchmarking tools and extract data from this prototype.

- **Phase 2:** In this phase, we will extend Phase 1 Protopype and add all necessary parameters. We will start by implementing the static paramenters, as we consider to be easier to implement and less probable to generate unexpected results, due to its static nature. After that, we will implement the dynamic parameters, which will require more time to implement and test. We will also implement the benchmarking tools and extract data from this prototype. Finally, we will compare the results

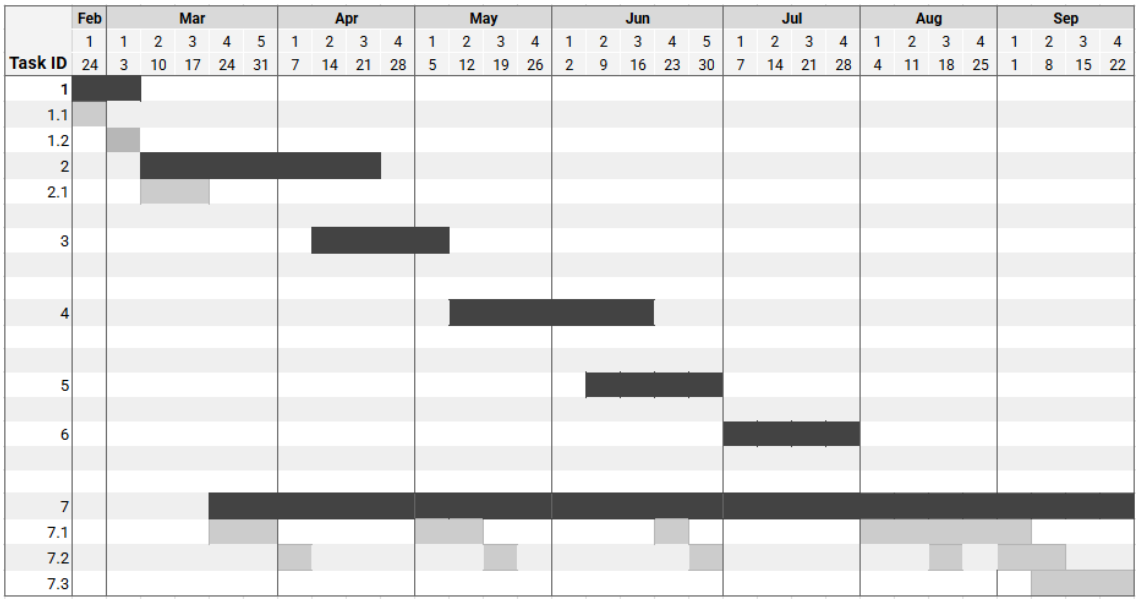

Figure 4.1: Gantt chart of the workplan

of the two prototypes and evaluate the impact of each parameter in the system's performance and what values do we consider to be the best for each parameter.

Upon completing these development phases, particular elements of the solution will be subject to thorough analysis and meticulous implementation:

- **Differential Privacy:** The main focus of this work is to provide a solution that is able to provide formal privacy guarantees to anonymization network's users. Formal privacy guarantees are a important aspect of our system, as it will provide a clear understanding of the privacy guarantees that users have when using our system. Therefore, we will have to implement the differential privacy mechanisms and evaluate its impact in the system's performance.

- **Flexible Configuration & Parameterization:** Another important aspect of the work we want to develop is the ability to provide flexible behaviour to the system. This flexibility can be divided into two main subjects: the users, and the correspondent needs and preferences, are one source of flexibility and we consider important to provide a system that can be easily configured by the user and fit his needs. On the other hand, the system itself should be able to adapt to the real-time conditions that it is subject to. For example, the system should be able to adapt to the network's jitter and latency, and adapt the noise added to the messages accordingly, by modifying the privacy parameters. Therefore, our system will have a flexible configuration and parameterization, and a vast number of parameters lead to a vast range of tests that we will have to perform.

4.2 Tasks & Milestones

Task ID	Description	Dependency
1	**Revision & Consilidation of Elaboration Guidelines**	
1.1	System Model & Evaluation	-
1.2	Initial Specification of Components	1.1
2	**Phase 1 Development & Prototyping**	
2.1	Setup Development Environment	-
2.2	Development of the first prototype	2.1
3	**Phase 1 Evaluation & Validation**	
3.1	Development of the first tests	2.1
3.2	Testing the Phase 1 Prototype	3.1
4	**Phase 2 Development & Prototyping**	
4.1	Extending the first prototype	3.2
5	**Final Validation & Experimental Evaluation**	
5.1	Testing the Final Prototype	4.1
5.2	Optimizations and Final validations	4.1
5.2	Final Observations	5.2, 5.3
6	**Dissertation Report**	
6.1	Report Writing	-
6.2	Report Revision	-
6.3	Preparation for Presentation	-

Table 4.1: Overview of tasks and their dependencies as presented in Figure 4.1

Bibliography

[0] L. von Ahn, A. Bortz, and N. J. Hopper. "k-anonymous message transmission". In: *Proceedings of the 10th ACM Conference on Computer and Communications Security*. CCS '03. Washington D.C., USA: Association for Computing Machinery, 2003, 122–130. ISBN: 1581137389. DOI: 10.1145/948109.948128. URL: https://doi.org/10.1145/948109.948128.

[0] M. Alimardani and S. Milan. "The Internet as a Global/Local Site of Contestation: The Case of Iran". In: *Global Cultures of Contestion*. Ed. by D. T. Della and G. K. Thomas. Springer, 2018, pp. 171–192. URL: https://link.springer.com/chapter/10.1007/978-3-319-78889-7_9.

[0] *Arti - The Tor Project*. https://gitlab.torproject.org/tpo/core/arti. 2020.

[0] S. Aryan, H. Aryan, and J. A. Halderman. "Internet Censorship in Iran: A First Look". In: *Proceedings of the 3rd USENIX Workshop on Free and Open Communications on the Internet (FOCI 13)*. USENIX Association, 2013. URL: https://www.usenix.org/conference/foci13/workshop-program/presentation/aryan.

[0] L. Barman et al. "Groove: Flexible Metadata-Private Messaging". In: *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, 2022-07, pp. 735–750. ISBN: 978-1-939133-28-1. URL: https://www.usenix.org/conference/osdi22/presentation/barman.

[0] D. Barradas et al. "Poking a Hole in the Wall: Efficient Censorship-Resistant Internet Communications by Parasitizing on WebRTC". In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2020, pp. 35–48. DOI: 10.1145/3372297.3423345. URL: https://doi.org/10.1145/3372297.3423345.

[0] A. Campan and T. Truta. "Data and Structural k-Anonymity in Social Networks". In: vol. 5456. 2008-01, pp. 33–54. ISBN: 978-3-642-01717-9. DOI: 10.1007/978-3-642-01718-6_4.

[0] S. Chakravarty et al. "On the Effectiveness of Traffic Analysis against Anonymity Networks Using Flow Records". In: *Passive and Active Measurement*. Ed. by M. Faloutsos and A. Kuzmanovic. Cham: Springer International Publishing, 2014, pp. 247–257.

[0] D. Chaum. "Untraceable Electronic Mail, Return Addresses and Digital Pseudonyms". In: *Secure Electronic Voting*. Ed. by D. A. Gritzalis. Boston, MA: Springer US, 2003, pp. 211–219. ISBN: 978-1-4615-0239-5. DOI: 10.1007/978-1-4615-0239-5_14. URL: https://doi.org/10.1007/978-1-4615-0239-5_14.

[0] G. Cherubin, R. Jansen, and C. Troncoso. "Online Website Fingerprinting: Evaluating Website Fingerprinting Attacks on Tor in the Real World". In: *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, 2022-08, pp. 753–770. ISBN: 978-1-939133-31-1. URL: https://www.usenix.org/conference/usenixsecurity22/presentation/cherubin.

[0] H. Corrigan-Gibbs, D. Boneh, and D. Mazieres. "Riposte: An anonymous messaging system handling millions of users". In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2015, pp. 321–338.

[0] G. Danezis, R. Dingledine, and N. Mathewson. "Mixminion: design of a type III anonymous remailer protocol". In: *2003 Symposium on Security and Privacy, 2003.* 2003, pp. 2–15. DOI: 10.1109/SECPRI.2003.1199323.

[0] G. Danezis. "Statistical Disclosure Attacks". In: *Security and Privacy in the Age of Uncertainty*. Ed. by D. Gritzalis et al. Boston, MA: Springer US, 2003, pp. 421–426. ISBN: 978-0-387-35691-4.

[0] R. Dingledine, N. Mathewson, and P. Syverson. "Tor: The Second-Generation Onion Router". In: *Proceedings of the 13th USENIX Security Symposium*. USENIX Association, 2004, pp. 303–320. URL: https://www.usenix.org/conference/usenixsecurity04/tor-second-generation-onion-router.

[0] C. Dwork. "Differential privacy". In: *Proceedings of the 33rd International Conference on Automata, Languages and Programming - Volume Part II*. ICALP'06. Venice, Italy: Springer-Verlag, 2006, 1–12. ISBN: 3540359079. DOI: 10.1007/11787006_1. URL: https://doi.org/10.1007/11787006_1.

[0] C. Dwork and A. Roth. "The Algorithmic Foundations of Differential Privacy". In: *Found. Trends Theor. Comput. Sci.* 9.3–4 (2014-08), 211–407. ISSN: 1551-305X. DOI: 10.1561/0400000042. URL: https://doi.org/10.1561/0400000042.

[0] C. Dwork et al. "Calibrating noise to sensitivity in private data analysis". In: *Proceedings of the Third Conference on Theory of Cryptography*. TCC'06. New York, NY: Springer-Verlag, 2006, 265–284. ISBN: 3540327312. DOI: 10.1007/11681878_14. URL: https://doi.org/10.1007/11681878_14.

[0] Úlfar Erlingsson, V. Pihur, and A. Korolova. "RAPPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response". In: *Proceedings of the 21st ACM Conference on Computer and Communications Security*. Scottsdale, Arizona, 2014. URL: https://arxiv.org/abs/1407.6981.

[0] G. Figueira, D. Barradas, and N. Santos. "Stegozoa: Enhancing WebRTC Covert Channels with Video Steganography for Internet Censorship Circumvention". In: *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*. ASIA CCS '22. Nagasaki, Japan: Association for Computing Machinery, 2022, 1154–1167. ISBN: 9781450391405. DOI: 10.1145/3488932.3517419. URL: https://doi.org/10.1145/3488932.3517419.

[0] D. Goldschlag, M. Reed, and P. Syverson. "Hiding Routing Information". In: 1996-08. ISBN: 978-3-540-61996-3. DOI: 10.1007/3-540-61996-8_37.

[0] A. Gosain and N. Chugh. "Privacy Preservation in Big Data". In: *International Journal of Computer Applications* 100 (2014-08), pp. 44–47. DOI: 10.5120/17619-8322.

[0] J. van den Hooff et al. "Vuvuzela: scalable private messaging resistant to traffic analysis". In: *Proceedings of the 25th Symposium on Operating Systems Principles*. SOSP '15. Monterey, California: Association for Computing Machinery, 2015, 137–152. ISBN: 9781450338349. DOI: 10.1145/2815400.2815417. URL: https://doi.org/10.1145/2815400.2815417.

[0] N. Hopper and E. Y. Vasserman. "On the effectiveness of k;-anonymity against traffic analysis and surveillance". In: *Proceedings of the 5th ACM Workshop on Privacy in Electronic Society*. WPES '06. Alexandria, Virginia, USA: Association for Computing Machinery, 2006, 9–18. ISBN: 1595935568. DOI: 10.1145/1179601.1179604. URL: https://doi.org/10.1145/1179601.1179604.

[0] A. Houmansadr et al. "IP over Voice-over-IP for censorship circumvention". In: *CoRR* abs/1207.2683 (2012). arXiv: 1207.2683. URL: http://arxiv.org/abs/1207.2683.

[0] A. Inc. *Learning with Privacy at Scale*. URL: https://machinelearning.apple.com/research/learning-with-privacy-at-scale.

[0] M. Jakobsson. "Flash mixing". In: *Proceedings of the Eighteenth Annual ACM Symposium on Principles of Distributed Computing*. PODC '99. Atlanta, Georgia, USA: Association for Computing Machinery, 1999, 83–89. ISBN: 1581130996. DOI: 10.1145/301308.301333. URL: https://doi.org/10.1145/301308.301333.

[0] M. Jakobsson and A. Juels. "An optimally robust hybrid mix network". In: *Proceedings of the Twentieth Annual ACM Symposium on Principles of Distributed Computing*. PODC '01. Newport, Rhode Island, USA: Association for Computing Machinery, 2001, 284–292. ISBN: 1581133839. DOI: 10.1145/383962.384046. URL: https://doi.org/10.1145/383962.384046.

[0] R. Jansen and N. Hopper. "Shadow: Running Tor in a Box for Accurate and Efficient Experimentation". In: *Network and Distributed System Security Symposium*. 2012.

[0] R. Jansen and N. J. Hopper. "Shadow: Running Tor in a Box for Accurate and Efficient Experimentation". In: *Computer Science Engineering (CSE) Technical Reports*. 11-020. 2011.

[0] R. Jansen, J. Newsome, and R. Wails. "Co-opting Linux Processes for High-Performance Network Simulation". In: *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. Carlsbad, CA: USENIX Association, 2022-07, pp. 327–350. ISBN: 978-1-939133-29-52. URL: https://www.usenix.org/conference/atc22/presentation/jansen.

[0] R. Jansen, T. Vaidya, and M. Sherr. "Point Break: A Study of Bandwidth Denial-of-Service Attacks against Tor". In: *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, 2019-08, pp. 1823–1840. ISBN: 978-1-939133-06-9. URL: https://www.usenix.org/conference/usenixsecurity19/presentation/jansen.

[0] R. Jansen et al. "KIST: Kernel-Informed Socket Transport for Tor". In: *ACM Trans. Priv. Secur.* 22.1 (2018-12). ISSN: 2471-2566. DOI: 10.1145/3278121. URL: https://doi.org/10.1145/3278121.

[0] A. Jerichow et al. "Real-time mixes: a bandwidth-efficient anonymity protocol". In: *IEEE J.Sel. A. Commun.* 16.4 (2006-09), 495–509. ISSN: 0733-8716. DOI: 10.1109/49.668973. URL: https://doi.org/10.1109/49.668973.

[0] A. Johnson et al. "Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries". In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2013. DOI: 10.1145/2508859.2516651. URL: https://doi.org/10.1145/2508859.2516651.

[0] D. Kesdogan, J. Egner, and R. Büschkes. "Stop- and- Go-MIXes Providing Probabilistic Anonymity in an Open System". In: vol. 1525. 1998-04, pp. 83–98. ISBN: 978-3-540-65386-8. DOI: 10.1007/3-540-49380-8_7.

[0] D. Lazar, Y. Gilad, and N. Zeldovich. "Karaoke: Distributed private messaging immune to passive traffic analysis". In: *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 2018, pp. 711–725.

[0] N. Mathewson and R. Dingledine. "Practical Traffic Analysis: Extending and Resisting Statistical Disclosure". In: *Privacy Enhancing Technologies*. Ed. by D. Martin and A. Serjantov. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 17–34. ISBN: 978-3-540-31960-3.

[0] S. Matic, C. Troncoso, and J. Caballero. "Dissecting Tor Bridges: A Security Evaluation of their Private and Public Infrastructures". In: *Network and Distributed System Security Symposium*. 2017. URL: https://api.semanticscholar.org/CorpusID:28430784.

[0] U. Moeller. "Mixmaster Protocol Version 2". In: 2004. URL: https://api.semanticscholar.org/CorpusID:215824742.

[0] M. Nasr, A. Bahramali, and A. Houmansadr. "DeepCorr: Strong Flow Correlation Attacks on Tor Using Deep Learning". In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. Toronto, ON, Canada: ACM, 2018, pp. 1962–1976. DOI: 10.1145/3243734.3243858.

[0] J. P. Near and C. Abuah. *Programming Differential Privacy*. Vol. 1. 2021. URL: https://programming-dp.com/.

[0] N. Nipane, I. Dacosta, and P. Traynor. ""Mix-in-Place" anonymous networking using secure function evaluation". In: *Proceedings of the 27th Annual Computer Security Applications Conference*. ACSAC '11. Orlando, Florida, USA: Association for Computing Machinery, 2011, 63–72. ISBN: 9781450306720. DOI: 10.1145/2076732.2076742. URL: https://doi.org/10.1145/2076732.2076742.

[0] R. Nithyanand, X. Cai, and R. Johnson. "Glove: A Bespoke Website Fingerprinting Defense". In: *Proceedings of the 13th Workshop on Privacy in the Electronic Society*. WPES '14. Scottsdale, Arizona, USA: Association for Computing Machinery, 2014, 131–134. ISBN: 9781450331487. DOI: 10.1145/2665943.2665950. URL: https://doi.org/10.1145/2665943.2665950.

[0] V. Nunes et al. "Enhancing the Unlinkability of Circuit-Based Anonymous Communications with k-Funnels". In: *Proceedings of the ACM on Networking* 1.CoNEXT3 (2023), Article 18. DOI: 10.1145/3629140. URL: https://doi.org/10.1145/3629140.

[0] S. E. Oh et al. "DeepCoFFEA: Improved Flow Correlation Attacks on Tor via Metric Learning and Amplification". In: *Proceedings of the IEEE Symposium on Security and Privacy*. San Francisco, CA, USA: IEEE, 2022, pp. 1915–1932. DOI: 10.1109/SP46214.2022.9833742.

[0] M. Ohkubo and M. Abe. "A Length-Invariant Hybrid Mix". In: *Advances in Cryptology - ASIACRYPT 2000, 6th International Conference on the Theory and Application of Cryptology and Information Security, Kyoto, Japan, December 3-7, 2000, Proceedings*. Vol. 1976. Lecture Notes in Computer Science. Springer, 2000, pp. 178–191. DOI: 10.1007/3-540-44448-3_14.

[0] H. Pereira. "MIRACE: Multi-Path Integrated Routing Architecture for Censorship Evasion". MA thesis. NOVA School of Science and Technology, 2024.

[0] A. M. Piotrowska et al. "The Loopix Anonymity System". In: *26th USENIX Security Symposium (USENIX Security 17)*. Vancouver, BC: USENIX Association, 2017-08, pp. 1199–1216. ISBN: 978-1-931971-40-9. URL: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/piotrowska.

[0] T. Project. *Circumvention*. URL: https://tb-manual.torproject.org/circumvention/.

[0] T. Project. *Tor Plugable Transports*. URL: https://2019.www.torproject.org/docs/pluggable-transports.

[0] M. S. Rahman et al. "Tik-Tok: The Utility of Packet Timing in Website Fingerprinting Attacks". In: *Proceedings on Privacy Enhancing Technologies* 2020.3 (2020), pp. 5–24. DOI: 10.2478/popets-2020-0032.

[0] A. Sabzi et al. "NetShaper: A Differentially Private Network Side-Channel Mitigation System". In: *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, 2024-08, pp. 3385–3402. ISBN: 978-1-939133-44-1. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/sabzi.

[0] T. Shen et al. " DAENet: Making Strong Anonymity Scale in a Fully Decentralized Network ". In: *IEEE Transactions on Dependable and Secure Computing* 19.04 (2022-07), pp. 2286–2303. ISSN: 1941-0018. DOI: 10.1109/TDSC.2021.3052831. URL: https://doi.ieeecomputersociety.org/10.1109/TDSC.2021.3052831.

[0] Y. Shi and K. Matsuura. "Fingerprinting Attack on the Tor Anonymity System". In: *Information and Communications Security*. Ed. by S. Qing, C. J. Mitchell, and G. Wang. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 425–438.

[0] V. Shmatikov and M.-H. Wang. "Timing Analysis in Low-Latency Mix Networks: Attacks and Defenses". In: vol. 4189. 2006-01, pp. 18–33. ISBN: 978-3-540-44601-9. DOI: 10.1007/11863908_2.

[0] P. Sirinam et al. "Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning". In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. CCS '18. Toronto, Canada: Association for Computing Machinery, 2018, 1928–1943. ISBN: 9781450356930. DOI: 10.1145/3243734.3243768. URL: https://doi.org/10.1145/3243734.3243768.

[0] P. Sirinam et al. "Triplet Fingerprinting: More Practical and Portable Website Fingerprinting with N-shot Learning". In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. CCS '19. London, United Kingdom: Association for Computing Machinery, 2019, 1131–1148. ISBN: 9781450367479. DOI: 10.1145/3319535.3354217. URL: https://doi.org/10.1145/3319535.3354217.

[0] Y. Sun et al. "RAPTOR: routing attacks on privacy in tor". In: *Proceedings of the 24th USENIX Conference on Security Symposium*. SEC'15. Washington, D.C.: USENIX Association, 2015, 271–286. ISBN: 9781931971232.

[0] L. Sweeney. "k-anonymity: a model for protecting privacy". In: *Int. J. Uncertain. Fuzziness Knowl.-Based Syst.* 10.5 (2002-10), 557–570. ISSN: 0218-4885. DOI: 10.1142/S0218488502001648. URL: https://doi.org/10.1142/S0218488502001648.

[0] L. Sweeney. "Simple Demographics Often Identify People Uniquely". In: *Carnegie Mellon University, Data Privacy* (2000). URL: http://dataprivacylab.org/projects/identifiability/.

[0] C. Tang and I. Goldberg. "An improved algorithm for tor circuit scheduling". In: *Proceedings of the 17th ACM Conference on Computer and Communications Security*. CCS '10. Chicago, Illinois, USA: Association for Computing Machinery, 2010, 329–339. ISBN: 9781450302456. DOI: 10.1145/1866307.1866345. URL: https://doi.org/10.1145/1866307.1866345.

[0] *Tor - The Tor Project*. https://gitlab.torproject.org/tpo/core/tor. 2020.

[0] N. Tyagi et al. "Stadium: A Distributed Metadata-Private Messaging System". In: *Proceedings of the 26th Symposium on Operating Systems Principles*. SOSP '17. Shanghai, China: Association for Computing Machinery, 2017, 423–440. ISBN: 9781450350853. DOI: 10.1145/3132747.3132783. URL: https://doi.org/10.1145/3132747.3132783.

[0] A. Vilalonga, J. S. Resende, and H. Domingos. *TorKameleon: Improving Tor's Censorship Resistance with K-anonymization and Media-based Covert Channels*. 2023. arXiv: 2303.17544 [cs.CR]. URL: https://arxiv.org/abs/2303.17544.

[0] A. Vilalonga et al. "Algoritmos de Resposta Aleatória como Mecanismo para a Desvinculação entre o Tráfego de Entrada e de Saída em Sistemas de Anonimato Baseadas em Circuitos". In: *INForum* (2024). URL: https://www.inforum.pt/static/files/papers/INForum_2024_paper_77.pdf.

[0] T. Wang and I. Goldberg. "Walkie-Talkie: An Efficient Defense Against Passive Website Fingerprinting Attacks". In: *26th USENIX Security Symposium (USENIX Security 17)*. Vancouver, BC: USENIX Association, 2017-08, pp. 1375–1390. ISBN: 978-1-931971-40-9. URL: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/wang-tao.

[0] T. Wang et al. "Effective Attacks and Provable Defenses for Website Fingerprinting". In: *Proceedings of the 23rd USENIX Security Symposium*. San Diego, CA, USA: USENIX Association, 2014, pp. 143–156. URL: https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/wang_tao.

[0] S. L. Warner. "Randomized Response: A Survey Technique for Eliminating Evasive Answer Bias". In: *Journal of the American Statistical Association* 60.309 (1965). PMID: 12261830, pp. 63–69. DOI: 10.1080/01621459.1965.10480775. URL: https://www.tandfonline.com/doi/abs/10.1080/01621459.1965.10480775.

[0] Z. Weinberg et al. "StegoTorus: A camouflage proxy for the Tor anonymity system". In: 2012-10, pp. 109–120. DOI: 10.1145/2382196.2382211.

[0] P. Winter and S. Lindskog. "How the Great Firewall of China is Blocking Tor". In: *Proceedings of the 2nd USENIX Workshop on Free and Open Communications on the Internet (FOCI)*. USENIX Association, 2012. URL: https://www.usenix.org/conference/foci12/workshop-program/presentation/winter.

[0] D. I. Wolinsky et al. "Dissent in Numbers: Making Strong Anonymity Scale". In: *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. Hollywood, CA: USENIX Association, 2012-10, pp. 179–182. ISBN: 978-1-931971-96-6. URL: https://www.usenix.org/conference/osdi12/technical-sessions/presentation/wolinsky.

[0] C. V. Wright, S. E. Coull, and F. Monrose. "Traffic Morphing: An Efficient Defense Against Statistical Traffic Analysis". In: *Network and Distributed System Security Symposium*. 2009. URL: https://api.semanticscholar.org/CorpusID:2562331.

[0] X. Zhang et al. "Statistical Privacy for Streaming Traffic". In: *Proceedings 2019 Network and Distributed System Security Symposium* (2019). URL: https://api.semanticscholar.org/CorpusID:142503765.

[0] Y. Zhu et al. "Anonymity analysis of mix networks against flow-correlation attacks". In: *GLOBECOM '05. IEEE Global Telecommunications Conference, 2005.* Vol. 3. 2005, 5 pp.–. DOI: 10.1109/GLOCOM.2005.1577959.

[0] J. L. Zittrain et al. "The Shifting Landscape of Global Internet Censorship". In: Accessed: 2024-11-17. Berkman Klein Center for Internet & Society, 2017. URL: https://www.ssrn.com/abstract=2993485.